DevSync

Technical Documentation

Secure AI Project Collaborator: software architecture, deployment, components and technology stack of the Android application.

Document map

Section	What it answers
1. Software Architecture	How the code is organised (feature-first, Clean Architecture), how state flows (BLoC/Cubit), which cloud services are used, how configuration (`.env`) and security are handled.
2. Deployment	How the app and its backend are built, flavored (Free / Pro), obfuscated, signed, packaged as AAB/APK and released.
3. Component	Which building blocks exist (core kit, shared UI, feature modules, domain services) and how they collaborate.
4. Tech Stack	Every technology in use, why it was chosen, and the trade-offs of the free Spark plan.

> **How to read the status labels**
> Items marked **In repo** are visible in the current `lib/` structure (the `auth` feature and the `core/` folders). Items marked **Planned** come from the approved schema, requirements and feature specifications and follow the same conventions. They are drawn with dashed borders in the diagrams.

1. Software Architecture

1.1 Architectural overview

DevSync is a single-module Flutter application for Android. Its code is organised **feature-first**: every user-facing capability (authentication, profile, discovery and so on) owns its screens, state and data access, while everything reusable lives in a shared `core/` package. **Clean Architecture** is applied pragmatically: dependencies always point from the UI toward the data and then toward the infrastructure, never the other way. State is managed with **BLoC / Cubit**.

There is no custom server. The app talks directly to Firebase (Auth, Firestore, Cloud Messaging, Crashlytics, AI Logic) and to a small number of public HTTP APIs (GitHub REST and a file host). This keeps the project on the free Firebase **Spark** plan and shapes several decisions described later in this document.

1.2 Architecture diagram

Figure 1 shows the layers and their allowed dependencies. Solid borders exist in the repository today; the dashed box is planned.

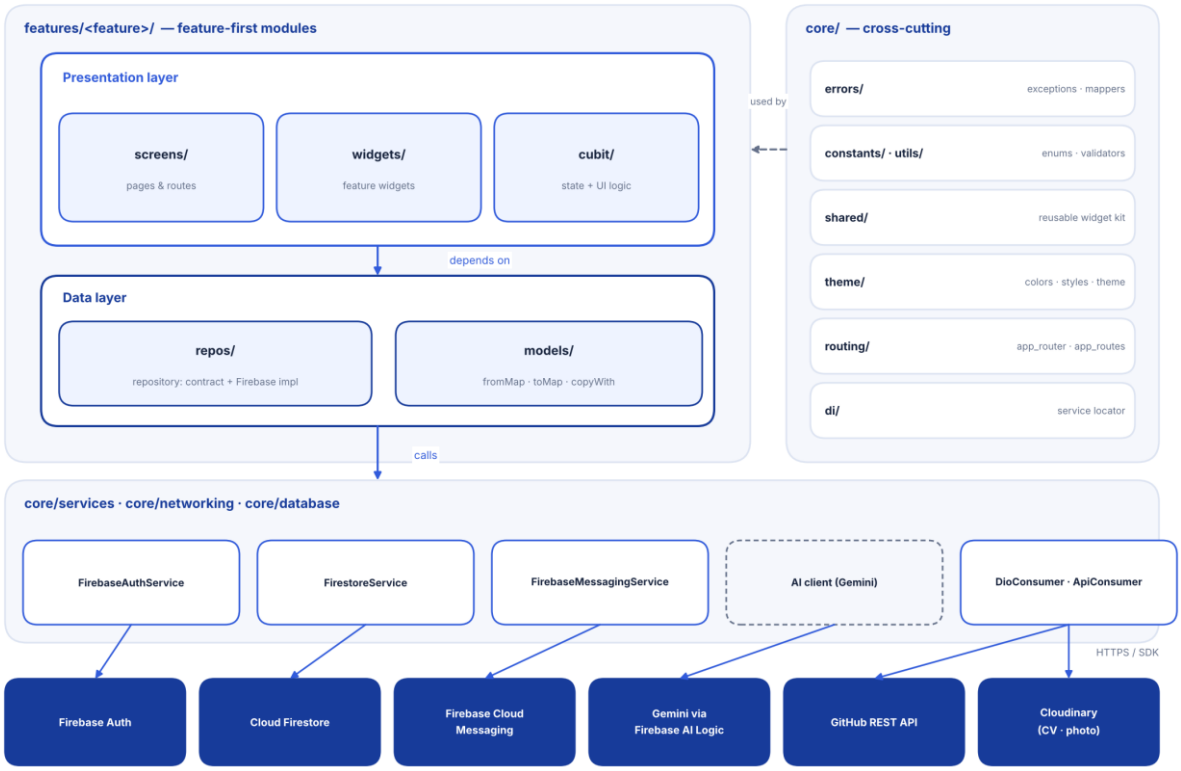


Figure 1. Layered, feature-first architecture of DevSync

Reading the diagram. A request starts in a screen or widget, is handled by a Cubit, goes through a repository in the data layer, and reaches an SDK or HTTP endpoint through a service wrapper in `core/services` or `core/networking`. The `core/` column on the right provides routing, theming, shared widgets, constants, utilities, error types and dependency injection to every feature.

1.3 Guiding principles

Principle	How it is applied in DevSync
Feature-first modules	Code is grouped by capability, not by technical type. Six developers can work on different features in parallel with few merge conflicts (planning risk R2).
Dependency rule	`presentation` → `data` → `core/services` → external SDKs. `core/` never imports from `features/`.
UI never touches Firebase	Widgets and Cubits depend on repositories only. Raw Firestore maps are converted to models inside the data layer (`fromMap`, `toMap`, `copyWith`).
Single source of UI truth	Cubits emit immutable states; widgets are a pure function of the current state.
Dependency injection	Services and repositories are registered once in `core/di` and injected through constructors, so they can be replaced by fakes in tests.
Fail gracefully	Errors are normalised in `core/networking/errors`. AI features degrade to cached or rule-based results instead of showing an empty screen (NFR-11).
Least privilege, no secrets	Firestore rules limit each user to their own data. Nothing secret is shipped in the app (NFR-03, NFR-04).

1.4 Project directory structure

The structure below mirrors the repository. The `auth` feature is the reference implementation; every new feature copies its `data/` and `presentation/` layout.

```
lib/
├── core/
│   ├── constants/      app_constants · app_icons · app_images · app_strings
│   ├── database/       cache · secure_data
│   ├── di/             service_locator · setup_locator
│   ├── networking/
│   │   ├── api/        api_consumer · dio_consumer · end_points
│   │   └── errors/     error_model · exceptions · firebase_error_handler
│   ├── routing/        app_router · app_routes
│   ├── services/
│   │   └── firebase/   firebase_auth_service · firebase_constants
│   │                   firebase_messaging_service · firestore_service
│   ├── shared/         animations · buttons · cards · chips · input_fields
│   │                   selectors · sheets · tab_bars
│   ├── static/
│   ├── theme/          app_colors · app_styles · app_theme
│   └── utils/          devsync_enums · validators
├── features/
│   └── auth/
│       ├── data/           models/ · repos/
│       └── presentation/   cubit/ · screens/ · widgets/
├── firebase_options.dart
└── main.dart
```

Folder	Responsibility	Rule
`core/constants`	App-wide strings, icon and image paths, configuration values read from `.env`.	No logic. No Flutter widgets.
`core/database`	Local persistence: `cache` for non-sensitive flags (for example `onboarding_seen`), `secure_data` for sensitive values.	Never store Firestore business data here.

Folder	Responsibility	Rule
`core/di`	`service_locator` holds the container, `setup_locator` registers services, repositories and Cubit factories at start-up.	Only place where concrete classes are constructed.
`core/networking/api`	`ApiConsumer` is the HTTP contract, `DioConsumer` implements it, `end_points` lists URLs.	Features call `ApiConsumer`, never `Dio` directly.
`core/networking/errors`	`ErrorModel`, typed exceptions, and `FirebaseErrorHandler` that turns Firebase error codes into readable messages.	Every failure that reaches a Cubit is already normalised.
`core/routing`	Named routes (`app_routes`) and the route table with guards (`app_router`).	Screens never build `MaterialPageRoute` by hand.
`core/services/firebase`	Thin wrappers over Firebase Auth, Firestore and Cloud Messaging plus `firebase_constants` (collection names, field keys).	No feature-specific logic.
`core/shared`	Reusable UI kit built from the design system (see Section 3.3).	Must read colors and styles from `core/theme`.
`core/theme`	`AppColors` (light and dark tokens), `AppStyles` (Inter typography), `AppTheme` (`ThemeData`).	No hard-coded colors in widgets.
`core/utils`	`devsync_enums` (track, level, purpose, statuses with `fromString`) and `validators`.	Pure Dart, easy to unit test.
`features/<name>/data`	`models` (one per Firestore collection) and `repos` (contract plus Firebase implementation).	Imports `core/services`, never Flutter widgets.
`features/<name>/presentation`	`cubit` (state and UI logic), `screens` (pages), `widgets` (feature-only components).	Imports repositories through the container only.
`main.dart`, `firebase_options.dart`	Start-up sequence; generated Firebase configuration.	Keep `main()` short and linear.

1.5 Layer responsibilities

Layer	Contains	May depend on	Must not
Presentation	Screens, feature widgets, Cubits and their states	Repositories (via DI), `core/shared`, `core/theme`, `core/routing`	Call Firebase or Dio directly; parse raw maps
Data	Repositories and models	`core/services`, `core/networking`, `core/utils`	Import widgets or Cubits
Core services	Auth, Firestore and Messaging wrappers; `DioConsumer`; cache	Firebase SDKs, Dio, platform plugins	Know about feature models
Core cross-cutting	Theme, shared UI, routing, DI, constants, utils, errors	Flutter, other core packages	Import anything from `features/`

1.6 State management: BLoC / Cubit

All screen state is managed with the `bloc` pattern. **Cubit** is the default because it exposes plain methods that emit states. A full **Bloc** (events and states) is reserved for flows where an audit trail of events is valuable. Figure 2 shows the unidirectional flow of a typical interaction.

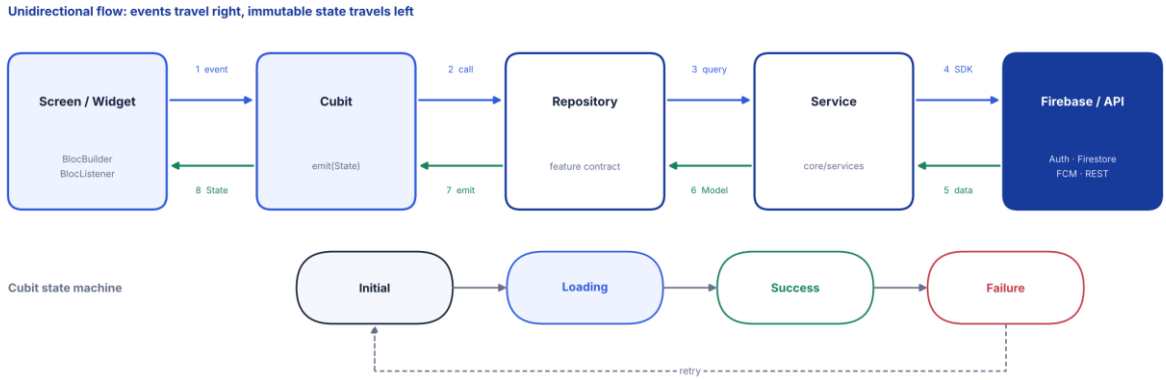


Figure 2. Event and state flow through a Cubit

Concern	Convention
Granularity	One Cubit per screen or flow, provided with `BlocProvider` at the route. Only session, theme and the notification badge are scoped globally.
States	Immutable and comparable (Equatable). Typical set: `Initial`, `Loading`, `Success`, `Failure`. Forms use one state object with a status field and `copyWith`.
Validation	Field validation runs in the Cubit using `core/utils/validators`, for example the required profile fields (name, track, level, at least one language).
Rendering vs side effects	`BlocBuilder` renders UI. `BlocListener` performs one-off effects: navigation, snackbars, bottom sheets.
Streams	Cubits that follow live data (notifications, request status) subscribe to a Firestore stream and cancel it in `close()`.
Errors	Repositories throw typed exceptions from `core/networking/errors`. The Cubit catches them and emits a `Failure` state with a user-friendly message.
Session and guest gate	`AuthCubit` exposes the session. When a guest taps an action that needs an account, a Login Required sheet appears and the original destination is stored so the user returns to the same screen after signing in.

The snippet below is illustrative; class names follow the files in `core/networking/errors` and `features/auth`.

```
class AuthCubit extends Cubit<AuthState> {
  AuthCubit(this._repo) : super(const AuthState.initial());
  final AuthRepo _repo;

  Future<void> signInWithEmail(String email, String password) async {
    emit(const AuthState.loading());
    try {
      final user = await _repo.signInWithEmail(email, password);
      emit(AuthState.success(user));
    } on AppException catch (e) {
      emit(AuthState.failure(e.message));
    }
  }
}
```

1.7 Backend and cloud services

The backend is entirely managed. The table lists each service, its role and where it is wrapped in the code.

Service	Role in DevSync	Where in code
Firebase Authentication	Sign-in with Google and Email through FirebaseUI (FR-01), session handling, logout and account deletion. GitHub is linked from inside the profile by username; it is not a sign-in method.	`firebase_auth_service`
Cloud Firestore	Primary database for profiles, projects, requests, invitations, teams, tasks, matches, feedback and notifications. Protected by security rules and composite indexes. Multi-document changes use transactions.	`firestore_service`, feature repos
Firebase Cloud Messaging (FCM)	Notification permission, device token, and handlers for foreground, background and terminated states. The payload `{type, refId, projectId}` is passed to the router, which opens the matching screen.	`firebase_messaging_service`, `app_router`
Firebase Crashlytics	Crash reporting and the crash-free-session KPI (95% target). Receives Dart symbol files for obfuscated builds.	`main.dart`
Firebase AI Logic (Gemini)	Explanations, skill-gap tips, join pitches and project set-up. Prompt templates are read from Firestore (`prompts/{version}`) so the active version can change without an app update.	AI client (planned)
GitHub REST API (public)	One call returns the user's public repositories with language and topics. No token is stored. Re-sync is throttled to about once per hour because the unauthenticated limit is about 60 requests per hour.	`DioConsumer`
Cloudinary (file host)	Stores the CV (PDF, max 5 MB) and profile photo through an unsigned upload preset. Only the returned URL is saved in Firestore, because Firebase Storage needs a paid plan.	`DioConsumer` (upload service planned)

> **Notifications on the Spark plan**
>
> Without Cloud Functions, the app cannot send a server-triggered push. The designed pipeline is: the app that causes an event writes a record to `notifications/{uid}/items`; the recipient's app listens to that collection and shows the bell badge, the list, and a local notification while the app is running. FCM is integrated on the client so that permission, token and message handling are ready.
>
> **Consequence:** there is no push when the app is fully closed. **Upgrade path:** on the Blaze plan, one Cloud Function triggered by a new `items` document can forward the same record through FCM without changing the client contract. This should be confirmed against requirement US-16 (delivery within 10 seconds in foreground, background and terminated states).

1.8 Data architecture

Firestore Schema v1.1 is the single source of truth. The collections are grouped below; field-level details stay in the schema document.

Group	Collections (paths)	Purpose
Identity and profile	`users/{uid}`, `users/{uid}/githubData/latest`, `users/{uid}/detectedSkills/{id}`, `users/{uid}/saved/{projectId}`, `userPrivate/{uid}`, `publicIds/{publicId}`	Public profile and matching input, GitHub analysis, AI-detected skills awaiting user approval, saved projects, private flags, unique short user ID.
Projects and collaboration	`projects/{id}`, `projects/{id}/interests/{uid}`, `requests/{projectId}_{applicantId}`, `invitations/{projectId}_{inviteeId}`	Project with track slots, Express Interest, Request to Join, owner invitations. Composite IDs block duplicates.
Teams and work	`teams/{projectId}`, `.../members/{uid}`, `.../tasks/{taskId}`, `.../messages/{id}`	Team roster, task board (todo, inProgress, inReview, done), optional team chat.
AI and quality	`matches/{uid}/items/{projectId}`, `feedback/{id}`, `prompts/{version}`, `config/prompts`, `aiUsage/{uid}`	Cached match results and Match History, user ratings of AI output, versioned prompts, daily soft usage limit.
Notifications	`notifications/{uid}/items/{id}`	In-app notification list written by the app that causes the event.

Conventions that every repository follows

- Field and collection names use `camelCase`; enum values are stored as lowercase camelCase (`inProgress`, `uiux`) and mapped by a `fromString` in `devsync_enums`.
- Every document has `createdAt`; edits set `updatedAt`. Optional fields are omitted instead of stored empty.
- Lists and cards store a small snapshot (name, photo) to avoid one extra read per row.
- Computed values (profile completion, match score) are calculated in the app and cached only in `matches`.
- Skills come from a catalog file in the app, not from Firestore. Free/Pro is a build-time setting, not data.

Transactions that protect integrity

Operation	What the transaction guarantees
Create profile	A unique `publicId` is reserved in `publicIds` together with the `users` document; on collision a new ID is generated.
Publish project	Validates tracks, fills the owner's slot, computes `teamSize`, `filledCount`, `trackKeys`, `openTrackKeys`, creates the team and owner member, converts `draftInvites` into invitations.
Accept request or invitation	Fails if the request is not pending or the track is full; updates the request, adds the member, increments `filled`, refreshes `openTrackKeys`, sets the project to `full` when every track is filled.
Leave team	Removes the member, decrements counters, reopens a `full` project. The owner cannot leave; they close the project.
Delete account	Removes profile data and memberships; open projects are set to `completed`. Files on the file host cannot be deleted (known limit).

1.9 Configuration and the .env file

Environment-specific values are kept out of source code in a `.env` file at the project root. The file is loaded once in `main()` before `runApp`, declared as an asset in `pubspec.yaml`, and **excluded from Git** through `.gitignore`. A committed `.env.example` lists every key with an empty value so a new developer can set up the project in minutes.

Rule	Detail
Single access point	Keys are read only in `core/constants` (`app_constants`, `end_points`) and exposed as typed constants. No other file calls the dotenv API.
Fail fast	If a required key is missing, start-up throws a clear error in debug builds instead of failing later inside a request.
Per environment	Developers keep their own `.env`. The release `.env` is supplied on the build machine and never pushed to the repository.

Example keys (finalise against app_constants)

Key	Purpose	Sensitivity
`CLOUDINARY_CLOUD_NAME`	Cloud used for CV and photo upload	Public identifier
`CLOUDINARY_UPLOAD_PRESET`	Unsigned preset that limits what can be uploaded	Low; abuse is limited by preset settings
`GITHUB_API_BASE_URL`	Base URL for GitHub REST calls	Public
`GITHUB_SYNC_THROTTLE_MINUTES`	Minimum time between GitHub re-syncs (about 60)	Public
`AI_DAILY_REQUEST_LIMIT`	Soft daily limit checked against `aiUsage` (value still to be decided)	Public
`MAX_CV_SIZE_MB`	Client-side upload validation (5)	Public

> **Security reality check**
>
> Anything bundled with the app, including a `.env` declared as an asset, can be extracted from the APK or AAB. **Code obfuscation does not encrypt assets.** The `.env` file is therefore for configuration, not for secrets. This matches the schema rule that nothing secret may be put in the app.
>
> Firebase API keys in `firebase_options.dart` are identifiers, not secrets; protect them with Firestore rules and by restricting the key to the app's package name and signing fingerprints. Signing credentials live in `key.properties` and the keystore, both git-ignored. If a feature ever needs a real secret, it must be called from a server (Blaze plan and Cloud Functions), not from the app. Passing values with `--dart-define-from-file` is harder to scrape than a plain-text asset but is not a vault either.

1.10 Error handling and resilience

Failure	Handling
Firebase error (permission denied, network, auth)	`FirebaseErrorHandler` maps the error code to an `ErrorModel` with a readable message; the Cubit emits `Failure`.
GitHub rate limit or failure	`DioConsumer` raises a typed exception; the user continues with manual profile data and sees a clear message (UC-01).
LLM slow or unavailable	The result is served from the `matches` cache or computed by the rule-based `MatchingService`; `matches.source` is `cached` or `fallback`. No empty screens (NFR-11).
Duplicate request or invitation	Composite document IDs and security rules reject the second attempt.
Track slot filled meanwhile	The accept transaction aborts and the user gets a specific message.
Offline	Reads use the Firestore local cache; writes show a clear offline message or are queued (UC-05).

1.11 Security architecture

Concern	Control	Requirement
Data access	Firestore rules with least privilege: users write only their own data; guests read `projects` only; team data is readable by members only; tests run on the Emulator.	NFR-03
Secrets	No keys or tokens in code, repo or `.env`; GitHub uses public data only; AI goes through Firebase AI Logic.	NFR-04
Android permissions	Runtime permissions (for example notifications) are requested at the moment of need only.	NFR-05
Reverse engineering	Release builds are obfuscated and shrunk (Section 2.4).	NFR-06
Privacy	Only data needed for matching is read; the user can disconnect GitHub at any time; AI-detected skills are never shown as verified facts.	NFR-07
Transport and storage	HTTPS for all calls; sensitive local values in `secure_data`.	Plan §5

2. Deployment

2.1 Deployment view

Figure 3 shows how source code becomes a signed, obfuscated release and where the running app connects. Everything on the right of the developer workstation is managed infrastructure; no server is operated by the team.

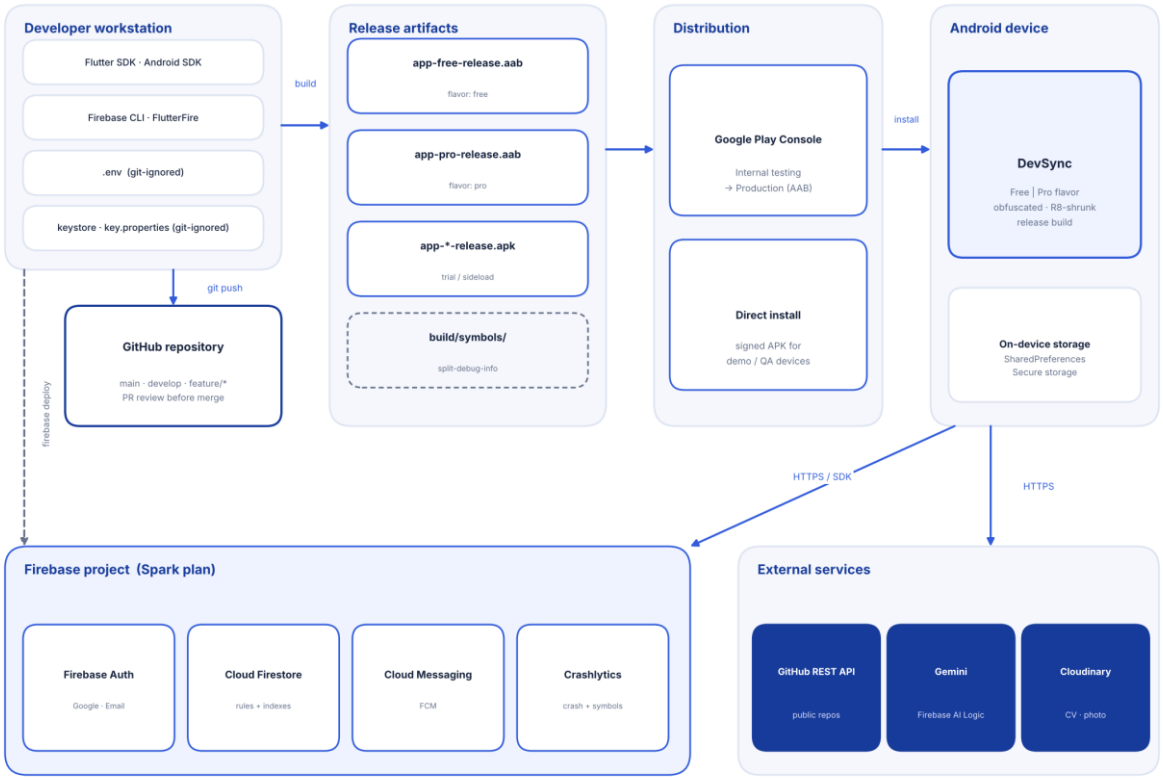


Figure 3. Build, distribution and runtime deployment

Node	Technology	Responsibility
Developer workstation	Flutter SDK, Android SDK, Firebase CLI, FlutterFire CLI	Runs builds, holds the git-ignored `.env`, keystore and `key.properties`, deploys Firestore rules and indexes.
GitHub repository	Git, branches `main`, `develop`, `feature/<module>-<name>`	Source of truth. No direct push to `main` or `develop`; every merge needs at least one reviewer.
Release artifacts	AAB (store), APK (trial), `build/symbols`	Outputs of the build for both flavors. Symbols are archived per version.
Distribution	Google Play Console, direct APK install	Internal testing and production for AAB; signed APK for demo and QA devices.
Android device	Android with the Free or Pro build	Runs the obfuscated release; stores flags in `SharedPreferences` and sensitive values in secure storage.
Firebase project (Spark)	Auth, Firestore, FCM, Crashlytics	Managed backend; rules and indexes are the only deployable backend assets.
External services	GitHub REST, Gemini via AI Logic, Cloudinary	Public data, AI generation and file hosting, all called over HTTPS.

2.2 Build flavors: Free and Pro

Free and Pro are two **flavors of the same codebase**. They differ only in which features are available. Payment processing is out of scope, so Pro simply unlocks features at build time. Plan details are shown as "Plan benefits to be confirmed" in the UI until requirements are final; no prices or usage limits are invented.

Aspect	Free	Pro
Gradle flavor	`free`	`pro`
Application ID	Base ID, for example `com.devsync.app`	Base ID plus suffix `.pro`, so both can be installed side by side
App label	DevSync	DevSync Pro
Firebase app	Own `google-services.json` in `android/app/src/free/`	Own `google-services.json` in `android/app/src/pro/`
Sign-in fingerprints	SHA-1 / SHA-256 of the signing key registered for the Free app	Registered separately for the Pro app (needed for Google sign-in)
Feature entitlements (proposed)	Basic profile, project discovery, basic matching	Advanced AI matching, detailed skill gap and skill analysis, advanced insights
Locked feature behavior	A Pro-only action opens the plan information screen; the user can continue the upgrade flow or return to available features	Feature opens normally

The flavor is read in Dart with the Flutter `appFlavor` value and exposed through one `isPro` getter, so feature gating is a single check instead of scattered string comparisons.

```
import 'package:flutter/services.dart' show appFlavor;

bool get isPro => appFlavor == 'pro';
```

Illustrative Gradle configuration (Kotlin DSL) for flavors, signing and shrinking:

```
android {
    flavorDimensions += "tier"
    productFlavors {
        create("free") {
            dimension = "tier"
            resValue("string", "app_name", "DevSync")
        }
        create("pro") {
            dimension = "tier"
            applicationIdSuffix = ".pro"
            resValue("string", "app_name", "DevSync Pro")
        }
    }
    signingConfigs {
        create("release") {
            val props = java.util.Properties().apply {
                load(rootProject.file("key.properties").inputStream())
            }
            storeFile = file(props["storeFile"] as String)
            storePassword = props["storePassword"] as String
            keyAlias = props["keyAlias"] as String
            keyPassword = props["keyPassword"] as String
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = true
            isShrinkResources = true
        }
    }
}
```

2.3 Android App Bundle (AAB) and APK generation

The AAB is the store format; the APK is used for direct installation on demo and QA devices. Both are built per flavor in release mode with obfuscation enabled.

```
# Store bundles (one per flavor)
flutter build appbundle --flavor free --release \
    --obfuscate --split-debug-info=build/symbols/free
flutter build appbundle --flavor pro --release \
    --obfuscate --split-debug-info=build/symbols/pro

# Trial APK for direct install
flutter build apk --flavor pro --release \
    --obfuscate --split-debug-info=build/symbols/pro-apk
```

Artifact	Output path	Use
Free bundle	`build/app/outputs/bundle/freeRelease/app-free-release.aab`	Google Play (internal testing, then production)
Pro bundle	`build/app/outputs/bundle/proRelease/app-pro-release.aab`	Google Play
Trial APK	`build/app/outputs/flutter-apk/app-pro-release.apk`	Demo and QA devices; signed with the same release key
Debug symbols	`build/symbols/<flavor>/`	De-obfuscating stack traces; archive per version

Signing

- Create an upload keystore once with `keytool` and store it outside the repository; keep a secure backup, because losing it blocks updates.
- Reference it from `android/key.properties` (git-ignored) as shown in the Gradle snippet.
- Enable Play App Signing so Google holds the final app-signing key and the team only manages the upload key.
- Increase the build number in `pubspec.yaml` (`version: x.y.z+build`) for every upload; Play rejects a repeated version code.

2.4 Code obfuscation and shrinking

Release builds apply two independent protections. **Dart obfuscation** renames classes, methods and fields in the compiled Dart code; `--split-debug-info` moves the symbol tables out of the binary and also reduces its size. **R8** (enabled with `isMinifyEnabled` and `isShrinkResources`) shrinks and obfuscates the Android and Java layer and removes unused resources. Both protections address NFR-06.

Flag or setting	Effect
`--obfuscate`	Renames Dart identifiers in the release build.
`--split-debug-info=<dir>`	Writes the symbol files to `<dir>`; required together with `--obfuscate`.
`isMinifyEnabled = true`	Turns on R8 code shrinking and obfuscation for Java/Kotlin code.
`isShrinkResources = true`	Removes unused Android resources from the package.
`proguard-rules.pro`	Keep rules for any library that needs reflection; add only when a release-only crash appears.

Symbols and crash reports

Obfuscated stack traces are unreadable without the matching symbol files, so they are handled as release assets:

- Keep `build/symbols/<flavor>` for every released version, outside the Git repository.
- Upload them to Crashlytics: `firebase crashlytics:symbols:upload --app=<ANDROID_APP_ID> build/symbols/<flavor>`.
- Read a trace locally with `flutter symbolize -i trace.txt -d build/symbols/<flavor>/app.android-arm64.symbols`.
- Verify obfuscation before each submission by opening the APK in a decompiler and confirming that class and method names are mangled.

> **Test the release build, not only debug**
> Obfuscation and R8 problems appear only in release mode. Run the full MVP path on at least two real devices with the release APK: login, profile, GitHub import, discovery, request to join, notification, accept, team tasks (NFR-12). Planning risk R9 requires the first trial release build before 20 November.

2.5 Release pipeline

#	Step	Gate
1	Freeze `develop` for the release; confirm all Must features are merged.	Flutter Lead
2	Bump the version and build number in `pubspec.yaml`.	Reviewed in PR
3	Open a pull request `develop` → `main`.	At least one review (NFR-13)
4	Run `flutter analyze` and the test suite; run Firestore rule tests on the Emulator.	All green
5	Build the Free and Pro AABs and the trial APK (Section 2.3).	Builds succeed
6	Verify obfuscation and smoke-test the release APK on two real devices.	QA hat
7	Upload the AABs to Play internal testing; upload symbols to Crashlytics.	Release lead
8	Tag `vX.Y.Z`, archive symbols and artifacts, update the README.	Release lead

2.6 Backend deployment (Firebase)

On the Spark plan the deployable backend assets are the Firestore rules and indexes; there are no Cloud Functions or Storage buckets to deploy.

Asset	Source file	Command or action
Security rules	`firestore.rules`	`firebase deploy --only firestore:rules`
Composite indexes	`firestore.indexes.json`	`firebase deploy --only firestore:indexes`
Rule tests	Emulator test suite (NFR-03 checklist)	`firebase emulators:start --only auth,firestore`
Prompt templates	`prompts/v1`, `config/prompts`	Created and switched in the Firebase console (admin only)
Demo data	15 to 20 projects, 5 to 10 test profiles	Seed script or console import before the demo
Client configuration	`firebase_options.dart`, per-flavor `google-services.json`	Generated with the FlutterFire CLI; re-run when an app is added

2.7 Release readiness checklist

Check	Evidence	Req.
Release built with `--obfuscate` and `--split-debug-info`	Build log; symbols archived	NFR-06
No keys or secrets in the repository	Repository scan; `.env`, keystore and `key.properties` ignored	NFR-04
Firestore rule checklist passes	Emulator test report	NFR-03
Runtime permissions requested only when needed	Manual review during testing	NFR-05
Free and Pro install side by side and show correct features	Two installs on one device	FR-23
Notification appears within 10 seconds	Timestamp comparison on real devices	NFR-02
Works on at least two real Android devices	Test log	NFR-12
95% crash-free sessions	Crashlytics dashboard	NFR-10
Symbols uploaded to Crashlytics	Readable stack trace from a test crash	NFR-06

3. Component

3.1 Component view

Figure 4 groups the application into feature modules, domain services and the `core/` infrastructure. Dependencies run top to bottom; feature modules never depend on each other directly. Cross-feature behavior goes through a shared service.

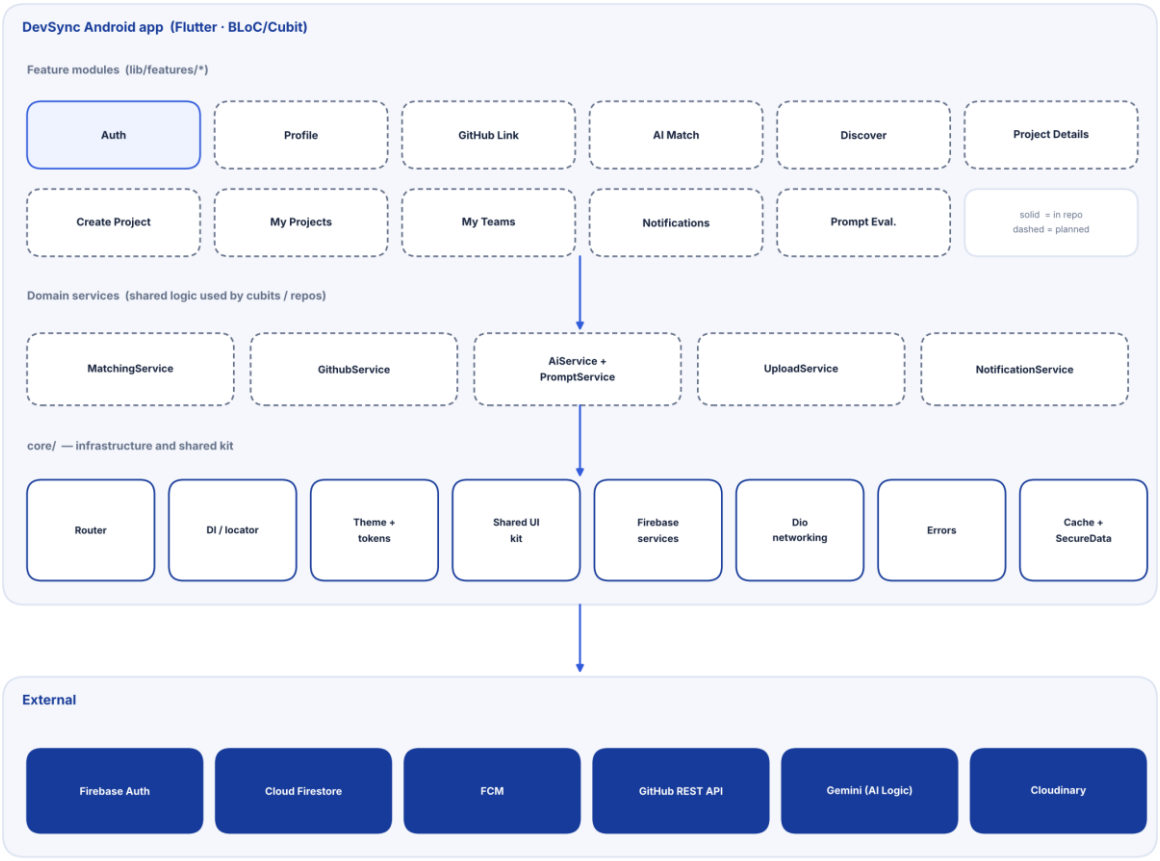


Figure 4. Component diagram (solid = in repo, dashed = planned)

3.2 Core components

Component	Location	Responsibility	Collaborates with
Bootstrap	`main.dart`, `firebase_options.dart`	Initialises Flutter bindings, `.env`, Firebase with generated options, dependency injection and Crashlytics; starts the app with the global providers.	DI, Firebase services
Dependency injection	`core/di`	Registers services, repositories and Cubit factories; single place that constructs concrete classes.	Every component
Routing	`core/routing`	Named routes, guards for the login gate, restoring the original destination, and opening screens from notification payloads and deep links.	AuthCubit, Messaging service

Component	Location	Responsibility	Collaborates with
Firebase services	`core/services/firebase`	`FirebaseAuthService`, `FirestoreService`, `FirebaseMessagingService`; `firebase_constants` centralises collection and field names.	Repositories
Networking	`core/networking/api`	`ApiConsumer` contract, `DioConsumer` implementation, `EndPoints` list.	GitHub and upload services
Error handling	`core/networking/errors`	`ErrorModel`, exceptions, `FirebaseErrorHandler`; converts technical failures into user messages.	Repositories, Cubits
Local storage	`core/database`	`cache` for non-sensitive flags; `secure_data` for sensitive values.	Onboarding, banner, session
Theme	`core/theme`	`AppColors` (light and dark tokens), `AppStyles` (Inter), `AppTheme` (`ThemeData`).	Shared UI kit
Shared UI kit	`core/shared`	Design-system widgets (Section 3.3).	All screens
Constants and utilities	`core/constants`, `core/utils`	Strings, icons, images, config from `.env`; enums with `fromString`; validators.	All layers

3.3 Shared UI component kit

The `core/shared` folders implement the DevSync Mobile design system. Blue is the only brand color; purple and violet are not allowed in light or dark mode. Typography is Inter, spacing follows an 8-point grid, and every touch target is at least 48 by 48 dp.

Folder	Components and specification
`buttons`	**Primary:** fill `#315BDB`, white text, 12 px radius, 48 px height, pressed `#2548B8`; dark mode fill `#6F91FF`. **Secondary:** transparent, 1.5 px outline in the primary color. **Text button:** borderless, primary text.
`input_fields`	48 px height, 12 px radius, fill `#F5F7FC`, 1 px border `#DDE4F2`. **Focused:** white fill, 1.5 px `#315BDB`. **Error:** white fill, 1.5 px `#C43D4B` with helper text 4 px below.
`chips`	**Skill chip:** 32 px pill; selected state uses `#EEF3FF` fill, `#315BDB` border and a leading check. **Status badges:** 24 px pills for Pending (warning), Accepted/Open (success), Declined/Full (error), and Purpose (Learning / Real-world).
`cards`	**Project card:** 16 px radius, 16 px padding, 1 px border; header with purpose badge and title; match box (for example "94% Match • Needs your Flutter experience"); footer with skill chips, capacity ("3/5 members") and timestamp. **Developer card:** 48 px avatar, name, availability pill, bio snippet, role, chips.
`selectors`	Segmented control (`#F5F7FC` container, 12 px radius, white active segment with border); pickers for track, level and task status.
`sheets`	Bottom sheets and dialogs with 16 px radius: Login Required, AI Feedback (helpful / not helpful, clarity 1 to 5), filters.
`tab_bars`	Bottom navigation with exactly five tabs (Discover, Community, My Projects, Messages, Profile), filled icon for the active tab; underline tabs with a 2 px primary indicator. Top app bar with the notification bell and unread dot.
`animations`	Implicit and explicit animations required by FR-24: Tween and Transition widgets, `AnimatedContainer`, `Hero`; used for onboarding transitions and the match-score progress.

Elevation uses flat tonal layers with 1 px borders (`#DDE4F2` light, `#303B51` dark) instead of heavy shadows. Icons are 2 px outline icons; filled icons mark the active navigation item only.

3.4 Feature modules

Each feature follows the layout of `features/auth`. Cubit names below are proposed and should be adjusted to the final code. Models follow Schema §10. Community and the chatbot are outside the current schema scope and have no collections.

Feature	Status	Cubits	Repositories and models	Firestore
Auth	In repo	`AuthCubit`	`AuthRepo`; user session	Firebase Auth; `users`
Profile	Planned	`ProfileSetupCubit`, `ProfileCubit`	`ProfileRepo`; `UserProfile`, `LanguageSkill`, `UserSkill`, `UserPrivate`	`users`, `userPrivate`, `publicIds`
GitHub integration	Planned	`GithubCubit`	`GithubRepo`; `GithubData`, `DetectedSkill`	`users/{uid}/githubData`, `detectedSkills`
AI Match	Planned	`MatchCubit`, `MatchHistoryCubit`	`MatchRepo`; `MatchResult`	`matches/{uid}/items`
Project discovery	Planned	`DiscoverCubit`, `FiltersCubit`	`ProjectRepo`; `Project`, `TrackSlot`	`projects`, `users/{uid}/saved`
Project details and join	Planned	`ProjectDetailsCubit`, `JoinRequestCubit`	`RequestRepo`; `JoinRequest`	`requests`, `projects/{id}/interests`
Create project	Planned	`CreateProjectCubit`	`ProjectRepo`; `DraftInvite`	`projects`, `teams`
My projects	Planned	`MyProjectsCubit`, `IncomingRequestsCubit`, `InvitationsCubit`	`InvitationRepo`; `Invitation`	`projects`, `requests`, `invitations`
My teams and tasks	Planned	`TeamsCubit`, `TaskBoardCubit`	`TeamRepo`; `Team`, `TeamMember`, `TeamTask`	`teams`, `members`, `tasks`
Notifications	Planned	`NotificationsCubit`	`NotificationRepo`; `AppNotification`	`notifications/{uid}/items`
Prompt evaluation	Planned	`FeedbackCubit`, `PromptReportCubit`	`FeedbackRepo`; `AiFeedback`, `PromptVersion`	`feedback`, `prompts`, `config`

3.5 Domain services

Domain services hold logic shared by more than one feature. They are plain Dart classes registered in the container and used by repositories or Cubits.

Service	Input → output	Notes and fallback
MatchingService	User profile and open projects → score (0 to 100), matched and missing skills, matched track	Order of factors: skill overlap, user track in `openTrackKeys`, level against difficulty, interests, purpose. Weights are still to be tuned on test profiles. Used as the rule-based fallback.
AiService / PromptService	Active prompt template plus context → explanation, learning tips, join pitch, project set-up	Calls Gemini through Firebase AI Logic; checks `aiUsage` first; stores the `promptVersion` with every result; falls back to cached or rule-based output.
GithubService	`githubUsername` → repositories, languages, topics, detected skills with confidence and evidence	One API call; throttled by `lastSync`; failure leaves the user on manual data. Detected skills stay `pending` until the user accepts.
UploadService	PDF or image file → hosted URL	Validates type and size (CV: PDF, 5 MB) before upload; stores only the URL; cannot delete from the host.
NotificationService	Domain event → notification record, stream, local notification, route payload	Writes to the recipient's `items`; listens through a Firestore stream; no push when the app is closed on Spark.
FeatureGate (plan)	Flavor → entitlement check	Single `isPro` source; opens the plan information screen when a feature is locked.

3.6 Component interaction example: Request to Join

#	Component	Action
1	ProjectDetailsScreen → AuthCubit	User taps Request to Join. A guest sees the Login Required sheet; after sign-in the router returns to the same project.
2	JoinRequestCubit → AiService	A pitch is generated from the profile and project; the user reviews and edits it.
3	RequestRepo → FirestoreService	Creates `requests/{projectId}_{applicantId}` with status `pending`; rules enforce one request, a track that is open, and the correct owner.
4	NotificationService	Writes a `request_received` record to the owner's notifications.
5	Owner's NotificationsCubit	The Firestore stream updates the bell badge and shows a local notification.
6	RequestRepo (transaction)	On accept: updates the request, adds the team member, increments `filled` and `filledCount`, refreshes `openTrackKeys`, sets the project to `full` if complete.
7	NotificationService → router	The applicant receives `request_accepted`; the payload opens the Team Workspace.

4. Tech Stack

4.1 Technology overview

The **Source** column shows where each item is established: the repository structure, Firestore Schema v1.1, or the project planning documents.

Layer	Technology	Usage in DevSync	Source
Client	Flutter and Dart, Material 3 base	Single Android UI codebase; theme built from design tokens	Plan
	BLoC / Cubit (`flutter_bloc`)	State management in every feature	Repo, Plan
	Service locator (`core/di`)	Dependency injection of services and repositories	Repo
	Routing (`app_router`, `app_routes`)	Named routes, login gate, deep links from notifications	Repo
	Dio (`DioConsumer`)	HTTP client for GitHub REST and file upload	Repo
	SharedPreferences and secure storage	Onboarding flag, banner state, sensitive values	Repo, Schema
	`.env` configuration	Non-secret, environment-specific settings	Project
	Inter font, DevSync design tokens	Typography and color system (blue only)	Design
	Tween, Transition widgets, `AnimatedContainer`, `Hero`	Implicit and explicit animations (FR-24)	Plan
Backend	Firebase Authentication (FirebaseUI)	Google and Email sign-in, session	Plan, Repo
	Cloud Firestore and security rules	Primary database, real-time streams, transactions	Schema
	Firebase Cloud Messaging	Notification permission, token and message handling	Repo, Plan
	Firebase Crashlytics	Crash reporting, crash-free KPI	Plan
	Firebase AI Logic (Gemini)	Matching explanations, tips, pitches, project set-up	Schema
	Firebase Emulator Suite	Testing security rules locally	Schema
Integrations	GitHub REST API (public)	Repositories, languages and topics for skill detection	Schema
	Cloudinary (unsigned upload)	CV and profile photo hosting; URL stored in Firestore	Schema
Build and release	Gradle product flavors (free, pro)	Two variants of one codebase	Plan
	Dart obfuscation, `--split-debug-info`, R8	Protection of release code (NFR-06)	Plan
	Android App Bundle and APK	Store bundle and trial APK	Plan
	Firebase CLI, FlutterFire CLI	Deploy rules and indexes; generate `firebase_options.dart`	Repo

Layer	Technology	Usage in DevSync	Source
Engineering	Git and GitHub, pull requests	Branches `main`, `develop`, `feature/*`; mandatory review	Plan
	GitHub Projects	Board: Backlog, In Progress, In Review, Done	Plan
	Figma and Stitch	Wireframes, mockups and screen generation	Plan

Package names are inferred from the repository structure and project documents. Confirm exact packages and versions in `pubspec.yaml` and `pubspec.lock` and add a version column before final submission.

4.2 Technology decisions and trade-offs

Decision	Why	Trade-off
Flutter for Android	One codebase and a team of six Flutter developers; fast UI iteration with the design system.	Android only for now; iOS is out of scope.
BLoC / Cubit	Explicit, testable state with a convention six developers can share.	More files than simple setState; mitigated by one folder layout per feature.
Feature-first with Clean Architecture	Parallel work and clear dependency direction.	Domain layer collapsed into data and services until needed.
Firebase on the Spark plan	No servers to operate, real-time streams, no cost.	No Cloud Functions or Storage; some logic runs in the client and limits are soft.
Public GitHub API without OAuth	No token to store, so no secret to leak.	About 60 requests per hour; public data only.
Gemini through Firebase AI Logic	No API key in the app; free tier.	Prompt templates are readable by signed-in users; quotas apply.
Cloudinary instead of Firebase Storage	Storage requires a paid plan.	File links are not protected by app rules and old files cannot be removed.
Build-time Free/Pro flavors	Shows the flavor requirement without payment processing.	Two bundles to build, test and publish.

4.3 Spark plan constraints and upgrade path

Limitation	Impact	Upgrade path (Blaze)
No Cloud Functions	No server-triggered push when the app is closed; AI limit and some checks are enforced in the app	Add a function on `notifications/{uid}/items` to send FCM; move AI quota checks server-side
No Firebase Storage	CV and photos live on an external host	Move uploads to Storage with owner-only rules
No place for secrets	Only public-data integrations are possible	Call private APIs and GitHub OAuth from functions
Client-side transactions	Integrity depends on rules and transactions in the app	Move accept, publish and delete flows to callable functions

4.4 Engineering conventions

Area	Convention
Branching	`main` for releases, `develop` for integration, `feature/<module>-<name>` for work. No direct push to `main` or `develop`.
Code review	Every pull request needs at least one review from the Flutter Lead or the module's backup (NFR-13).
Definition of Done	Works on a real device, passes review, merged to `develop`, documented in the README when needed.
Architecture checks	New features copy the `features/auth` layout; Cubits never import Firebase; widgets read colors from `AppColors`.
Documentation	Technical documentation and README follow best practice and are complete at the final milestone (NFR-14).